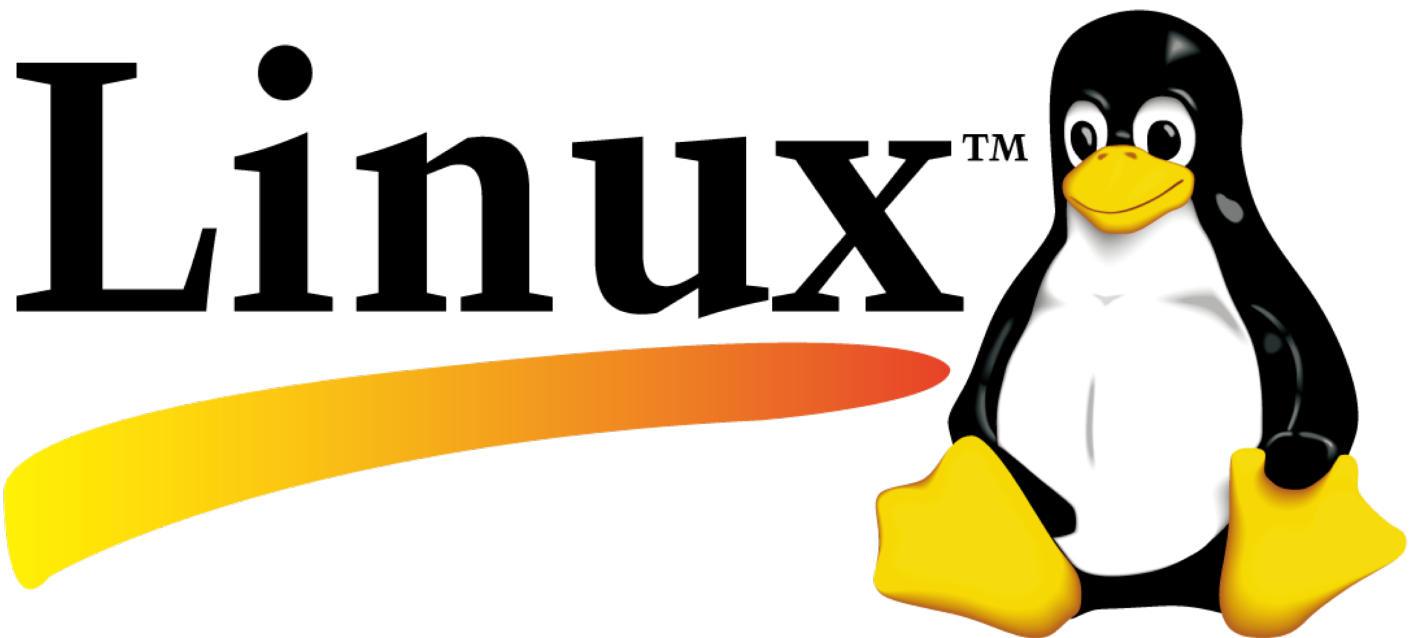




Appunti

Stacchiotti Elia

Indice

1	GitHub	1
1.1	Repository remoti	1
1.2	Creare e gestire repository remoti su GitHub	1
	Fonti	6

Listati

1

GitHub

In questo capitolo viene trattata la piattaforma *GitHub* e, più in particolare, la sua integrazione con il software *Git*, questo binomio è oggi ampiamente diffuso nello sviluppo software, soprattutto nel contesto open source. Come visto nel capitolo ??, *Git* è uno strumento per il versionamento dei file e fornisce meccanismi che si prestano naturalmente allo sviluppo collaborativo, tuttavia, *Git* opera in modo completamente locale e non fornisce direttamente strumenti per la condivisione e la sincronizzazione dei repository tra più utenti. In assenza di una piattaforma remota, la condivisione di un archivio richiederebbe infatti il trasferimento manuale dei dati tra macchine diverse (ad esempio tramite dispositivi fisici), rendendo difficile gestire in modo efficace il lavoro collaborativo.

GitHub nasce proprio per risolvere queste problematiche offrendo un'infrastruttura online che consente di ospitare repository *Git* e di renderli accessibili tramite rete. In questo modo la collaborazione tra più utenti diventa una caratteristica pienamente supportata grazie a strumenti che permettono la sincronizzazione delle modifiche, la gestione dei contributi e il coordinamento dello sviluppo.

1.1. Repository remoti

Con il termine *repository remoto* si indica, in modo generale, un archivio di file che si trova su una macchina diversa rispetto a quella su cui si sta lavorando. L'utilizzo di repository remoti risulta particolarmente utile in diversi scenari: da un lato permette di condividere un progetto tra più macchine o utenti, dall'altro consente di mantenere una copia di sicurezza dell'archivio in una posizione separata.

Nell'integrazione tra *Git* e *GitHub*, il repository remoto rappresenta una versione del repository *Git* ospitata su un server online, ovvero *GitHub*, e il suo scopo principale è quello di permettere la collaborazione tra più utenti. La condivisione, in questo contesto, non si limita alla semplice consultazione o al download dell'archivio, ma comprende anche la possibilità di modificarlo e aggiornare la versione condivisa. In termini operativi, il flusso di lavoro tipico prevede che un utente scarichi una copia del repository, apporti modifiche localmente utilizzando *Git* e successivamente invii tali modifiche al repository remoto, aggiornandone il contenuto. Sebbene questo processo possa apparire semplice, nella pratica emergono problematiche non banali, come la gestione di modifiche simultanee da parte di più utenti. *Git* e *GitHub* forniscono strumenti specifici per affrontare queste situazioni, evitando la perdita di dati e garantendo che le diverse versioni del progetto vengano integrate in modo coerente.

1.2. Creare e gestire repository remoti su GitHub

Per creare una repository remota su *GitHub* è sufficiente andare sulla pagina web di *GitHub* e creare un account, una volta nell'area personale basta premere il pulsante "New". Successivamente è possibile specificare il nome della repository e la tipologia, *public* o *private*: nel primo caso la repository è accessibile pubblicamente, mentre nel secondo è visibile e modificabile solo dagli utenti autorizzati, di default lo stesso che ha creato la repository. Una volta completata la creazione *GitHub* fornisce un URL che identifica univocamente la repository e che rappresenta l'elemento principale per interagire tramite *Git*. Sebbene *GitHub* metta a disposizione anche un editor web per creare, modificare ed eliminare file

direttamente dal browser, è consigliabile limitarne l'utilizzo, soprattutto nelle fasi iniziali, in quanto può introdurre modifiche non sincronizzate con il repository locale e rendere meno chiaro il flusso di lavoro.

Per iniziare a lavorare sulla repository remota, è possibile clonarla in locale tramite il comando `git clone <URL> <cartella>`, questo crea una copia completa della repository remota all'interno di una cartella locale (il cui percorso è da specificare in `<cartella>`, percorso relativo rispetto alla posizione in cui si trova avviata la sessione di Git Bash); nel caso di repository private potrebbe essere richiesto di effettuare l'autenticazione prima di completare l'operazione. Dopo il clone, il repository locale è già configurato per comunicare con quello remoto, infatti Git crea automaticamente un riferimento chiamato *origin*, che memorizza l'URL della repository remota. Banalmente, avendo l'URL, Git permette, attraverso comandi specifici, di effettuare richieste web al server GitHub che ospita la repository remota. Per verificare i repository remoti associati, è possibile utilizzare il comando `git remote -v` (si possono configurare più repository remoti per uno stesso repository locale ma questa è un'applicazione abbastanza avanzata) che mostra gli alias configurati e i relativi URL (gli alias sarebbero i nomi associati agli URL), in questo caso di esempio si dovrebbe avere un output di questo tipo:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/eseempio2
$ git remote -v
origin https://github.com/ViscidoSnake/eseempio2.git (fetch)
origin https://github.com/ViscidoSnake/eseempio2.git (push)
```

cioè un solo alias, *origin*, che Git utilizza per effettuare le operazioni di *push* e *fetch*, operazioni fondamentali per mantenere l'aggiornamento tra repository remoto e repository locale. In parole molto semplici l'operazione di *push* carica nella repository remota i commit avvenuti nella repository locale mentre il *fetch* serve per scaricare nella cartella locale i cambiamenti e riferimenti Git dalla repository remota verso quella locale. Le modalità con cui si possono caricare update avvenuti nella cartella locale sono molti e tutti attraverso il comando `git push <opzioni>`, il modo in assoluto più semplice è quello di aggiungere nuovi commit, senza quindi cambiare radicalmente la struttura di partenza del repository remoto (questo accade se si inizia ad usare per esempio `git reset`), per fare questo si usa il comando `git push <alias> <branch locale>` dove appunto alias rappresenta un URL che punta alla repository remota e branch sarebbe il nome del branch locale da caricare, questa operazione non carica semplicemente l'ultimo commit ma tutti gli ultimi commit avvenuti sul particolare branch. Volendo provare, una volta effettuato il clone, basta creare un file, generare il primo commit, e poi fare il push nella repository remota, aprendo GitHub dovrebbe comparire il nuovo file. Oltre a vedere il file è interessante guardare anche l'output del comando:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/eseempio2 (main)
$ git push origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 221 bytes | 221.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/ViscidoSnake/eseempio2.git
* [new branch]      main -> main
```

dopo le prime righe che indicano quanti file sono stati caricati e la dimensione di essi, l'ultima riga informa che è stato creato un nuovo branch, questa volta remoto, che porta lo stesso nome del branch appena *pushato* (in realtà, come poi si vede anche dal log, il nome del branch remoto creato ha come prefisso l'alias che la repository locale ha usato per il push, in questo caso *origin*) e in più i due sono perfettamente allineati cioè quindi le versioni dell'archivio locale e dell'archivio remoto sono identiche. Eseguendo `git log --all --branch` è possibile vedere graficamente quanto detto. Una cosa molto importante da tener presente è che se si continuano a creare commit in locale oppure anche creare branch, fin tanto che non si effettua un push nella repository remota, questa resta non sincronizzata e ciò lo si può vedere dal grafo generato da `git log --all --graph` dove il branch remoto resta indietro rispetto a quello locale, in riferimento all'esempio di prima:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/eseempio2 (main)
$ git log --all --graph
* commit 4c8f7df447945c27939e3e9a2720ad04499958a1 (HEAD -> main)
| Author: Elia <Elia>
| Date: Thu Mar 26 09:32:14 2026 +0100
```

```
|
|      modificato file listaSpesa, aggiunte
|
| * commit e194fd454fa71e2f2c111553acf8c37a993f2b06
| Author: Elia <Elia>
| Date:   Thu Mar 26 09:30:07 2026 +0100
|
|      modificato file note, aggiunto pollo
|
| * commit ce5262c27d169420db89519b0ffab163e0e8ce20
| Author: Elia <Elia>
| Date:   Thu Mar 26 09:29:11 2026 +0100
|
|      aggiunto file note
|
| * commit dfa0d0106007b2cd13181e664ecacc7cda569b84 (origin/main)
| Author: Elia <Elia>
| Date:   Thu Mar 26 09:12:08 2026 +0100
|
| initial commit
```

Un'altra questione importante riguarda lo scaricamento dell'archivio modificato da GitHub verso la repository locale, questo si rende necessario se, per vari motivi, la versione remota è maggiormente aggiornata rispetto a quella locale, in questi casi si utilizzano i comandi `git fetch --all` e `git pull <alias> <branch remoto>`. Partendo dal primo, questo scarica i file nella cartella `.git` e permette, dalla repository locale, di vedere la timeline aggiornata dell'archivio eseguendo il comando `git log --all --graph`, vedendo il grafo è possibile capire quali update effettivamente ha ricevuto la repository remota rispetto a quella locale, per applicare tali cambiamenti in termini di file si usa il comando `git pull`, questa operazione va fatta sempre con cautela perchè il pull del particolare ramo remoto va a fondersi implicitamente con i file contenuti nel ramo locale in cui Git si trova, è quindi fondamentale assicurarsi sempre di posizionarsi nello stesso branch da *riallineare* e anche avere un staging area e working directory pulite. Una cosa che può accadere è vedere dal log la presenza di un branch remoto non presente nella versione attuale della repository locale, qui è impossibile effettuare un pull diretto perchè il branch locale non esiste, si potrebbe crearlo a partire dal giusto commit (informazione che si legge dal grafo), spostarsi sul nuovo branch e poi fare il pull, tuttavia tutto questo può essere eseguito da un solo comando ovvero `git checkout -b <branch locale> <alias>/<branch remoto>`; di seguito un esempio del caso appena descritto, per simulare l'aggiunta di un branch sulla repository remota è stato usato l'editor di GitHub. Partendo dal grafo mostrato appena sopra, da GitHub è stato aggiunto un branch chiamato "marco" ed è stato aggiunto il file `festa.txt`, eseguendo ora il comando `git fetch`:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/esempio2 (main)
$ git fetch
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 988 bytes | 89.00 KiB/s, done.
From https://github.com/ViscidoSnake/esempio2
* [new branch]      marco      -> origin/marco
```

per vedere chiaramente quali nuovi branch sono presenti nella versione remota:

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/esempio2 (main)
$ git log --all --graph
* commit 5320f73bd053088bbc0e50eaf20d62218220454c (origin/marco)
| Author: ViscidoSnake <132934376+ViscidoSnake@users.noreply.github.com>
| Date:   Fri Mar 27 09:36:07 2026 +0100
|
|      creato il file festa.txt
|
| * commit e194fd454fa71e2f2c111553acf8c37a993f2b06 (HEAD -> main, origin/main)
| Author: Elia <Elia>
| Date:   Thu Mar 26 09:30:07 2026 +0100
|
```

```
|      modificato file note, aggiunto pollo
|
* commit ce5262c27d169420db89519b0ffab163e0e8ce20
| Author: Elia <Elia>
| Date:   Thu Mar 26 09:29:11 2026 +0100
|
|      aggiunto file note
|
* commit dfa0d0106007b2cd13181e664ecacc7cda569b84
Author: Elia <Elia>
Date:   Thu Mar 26 09:12:08 2026 +0100

initial commit
```

a questo punto allora eseguo il comando `git checkout -b marco origin/marco`, si può vedere che è stato creato un nuovo branch locale perfettamente allineato a quello remoto.

```
snake@DESKTOP-VGK2IC9 MINGW64 /x/esempio2 (main)
$ git checkout -b marco "origin/marco"
Switched to a new branch 'marco'
branch 'marco' set up to track 'origin/marco'.

snake@DESKTOP-VGK2IC9 MINGW64 /x/esempio2 (marco)
$ git log --all --graph
* commit 5320f73bd053088bbc0e50eaf20d62218220454c (HEAD -> marco, origin/marco)
| Author: ViscidoSnake <132934376+ViscidoSnake@users.noreply.github.com>
| Date:   Fri Mar 27 09:36:07 2026 +0100
|
|      creato il file festa.txt
|
* commit e194fd454fa71e2f2c111553acf8c37a993f2b06 (origin/main, main)
| Author: Elia <Elia>
| Date:   Thu Mar 26 09:30:07 2026 +0100
|
|      modificato file note, aggiunto pollo
|
* commit ce5262c27d169420db89519b0ffab163e0e8ce20
| Author: Elia <Elia>
| Date:   Thu Mar 26 09:29:11 2026 +0100
|
|      aggiunto file note
|
* commit dfa0d0106007b2cd13181e664ecacc7cda569b84
Author: Elia <Elia>
Date:   Thu Mar 26 09:12:08 2026 +0100

initial commit

snake@DESKTOP-VGK2IC9 MINGW64 /x/esempio2 (marco)
$ ll
total 3
-rw-r--r-- 1 snake 197121 17 Mar 27 09:46 festa.txt
-rw-r--r-- 1 snake 197121 26 Mar 27 09:26 listaSpesa.md
-rw-r--r-- 1 snake 197121 78 Mar 27 09:26 notal.txt
```

Va precisata una questione, l'utilizzo della repository remota e il modo di gestirla sopra descritti non sono chiaramente gli unici modi possibili, infatti non sono stati affrontati temi cruciali come che cosa accade se si modifica in modo diverso uno stesso commit e poi si effettua il push, oppure cosa accade se si eliminano branch dalla repository remota; per gestire queste dinamiche complesse è necessario approfondire lo studio dei comandi `git push` e `git pull` i quali supportano molte opzioni che ne modificano in modo radicale il comportamento. Oltre a questo in genere esistono delle regole implicite nella gestione dell'archivio remoto che evitano proprio il verificarsi di situazioni complicate da sistemare, le più comuni sono:

- evitare di riscrivere la cronologia dei commit già condivisi (ad esempio tramite `git reset` seguito da push forzati), in quanto ciò può creare inconsistenze tra le copie locali dei vari collaboratori;
- evitare di lavorare direttamente sul branch principale, riservandolo a versioni stabili del progetto;

- effettuare aggiornamenti frequenti (pull) prima di eseguire un push, in modo da ridurre la probabilità di conflitti;
- utilizzare branch separati per funzionalità diverse, così da isolare le modifiche e semplificare eventuali merge.

Questo per sottolineare che le modalità di gestione descritte nei paragrafi precedenti sono pensate per casi relativamente semplici, in cui l'obiettivo principale è mantenere la repository remota perfettamente allineata a quella locale. In questo contesto, anche la collaborazione tra più utenti segue una logica ben definita: ciascun collaboratore opera su un proprio branch, evitando che altri possano effettuare commit o operazioni di merge sugli stessi rami, così da ridurre al minimo possibili interferenze e semplificare la gestione complessiva dell'archivio.

Fonti

- [1] Website Name <OR> I. Surname, I. Surname e I. Surname. *Title of the Website*. 2000. URL: <https://example.com> (visitato il 24/12/2020).